

Textbook Chapter

This project is rooted in Chapter 6 (Optimization) of Heath's *Scientific Computing*, with connections to Chapter 5 (Nonlinear Equations) and Chapter 3 (Linear Least Squares).

Contributions of the Authors

Both authors contributed equally to this project. Tian organized and wrote the manuscript, designed the experiments, and generated the figures. Frank implemented the Gauss-Newton and Levenberg-Marquardt solvers, performed the numerical analysis, and wrote the manuscript.

Use of AI Tools

We used LLM for assistance with:

(1) edit and refine the text for clarity and readability, and (2) assist with coding tasks such as debugging and validation, improving documentation, and refactoring the implementation into reusable modules, as well as polishing figure styling.

All mathematical derivations, experimental design, results, and solver implementations were produced by the authors. Any LLM-generated suggestions were treated as drafts and were carefully reviewed, tested, and corrected by the authors before inclusion.

Other Sources

We received feedback from the course staff on our project proposal and discussed general project directions during office hours. We did not receive further assistance from individuals outside our team.

Analyzing Gauss-Newton and Levenberg-Marquardt in Nonlinear Least Squares: Conditioning, Robustness, and Convergence

Frank Liang Tian Xia

December, 2025

1. Introduction

Nonlinear least-squares problems arise naturally in parameter estimation when a physics-based model must be fit to noisy measurements, a setting that is classically formulated through residual minimization in numerical optimization [Hea18]. In this project we use a damped oscillation response as a concrete testbed, motivated by applications where one seeks damping and frequency information from time-series data.

We implement two classical solvers: Gauss-Newton (GN) [Wik25a] and the Levenberg-Marquardt (LM) method [Wik25c], and study how algorithmic choices affect convergence and robustness on the objective defined by the residual fit. Our experiments target three mechanisms: (E1) sensitivity to initialization, (E2) instability under ill-conditioning, and (E3) the effect of measurement noise on parameter recovery.

The main takeaway is practical: across all experiments, LM is consistently more robust: it converges from a broader set of initial guesses, remains stable under ill-conditioning, and behaves predictably as noise increases. GN can be efficient when started near the solution with a well-conditioned Jacobian, but it is highly sensitive to initialization and can degrade sharply when the normal-equation subproblem becomes ill-conditioned. When the problem is well-posed, the primary effect of noise is a reduction in estimation accuracy rather than a loss of stability.

2. Problem Setup

Many physical systems respond to a disturbance by oscillating and gradually returning to equilibrium. Examples include a ship’s roll after a wave impact, a vibrating beam after a tap, or an RLC circuit after a voltage step. In these settings, engineers often want to estimate key properties such as the damping rate and the dominant frequency. These parameters are directly linked to stability, energy dissipation, and system health. Given measured time-series

data that contain noise and possible sensor bias, parameter estimation naturally becomes a curve-fitting problem. To make our algorithm analysis concrete, we use this fitting task as a practical testbed for studying numerical behaviors.

From a physical model to a fitting function

A standard starting point is the linear, second-order differential equation for an underdamped oscillator [Wik25b],

$$my''(t) + c_dy'(t) + ky(t) = 0,$$

where m is an effective mass, c_d is a damping coefficient, and k is a stiffness constant. When the system is underdamped (i.e., $c_d^2 < 4mk$), the solution has the form of a sinusoid with an exponentially decaying envelope:

$$y(t) = Ae^{-\lambda t} \cos(\omega t + \phi).$$

In practice, measurements may not be centered perfectly at zero due to sensor bias, drift, or an imperfect reference level. To account for this, we include a constant offset term c , leading to the model used throughout this project:

$$f(t; \beta) = Ae^{-\lambda t} \cos(\omega t + \phi) + c, \quad \beta = (A, \lambda, \omega, \phi, c).$$

Nonlinear least-squares objective

Suppose we observe data $\{(t_i, y_i)\}_{i=1}^m$, where t_i is the sampling time and y_i is the measured response. We define the residual at each point as

$$r_i(\beta) = y_i - f(t_i; \beta),$$

and estimate β by minimizing the sum of squared residuals [Hea18]:

$$S(\beta) = \frac{1}{2} \sum_{i=1}^m r_i(\beta)^2.$$

This objective is nonlinear in β because the parameters appear inside the exponential and the cosine. As a result, we cannot solve for β in closed form, and we instead rely on iterative numerical methods such as Gauss-Newton. Each parameter in β has a clear physical meaning:

- A (amplitude): the initial oscillation magnitude.
- λ (decay rate): how quickly the oscillations die out; larger λ means faster damping.
- ω (angular frequency): determines the oscillation rate (in radians per unit time).
- ϕ (phase): shifts the oscillation in time and aligns the cosine with the initial condition.
- c (offset): the long-time baseline level; as $t \rightarrow \infty$, the model approaches c .

Together, these parameters summarize both the transient behavior (through A, λ, ω, ϕ) and the measurement baseline (through c). The goal of the project is to compute β reliably from noisy data and to analyze how algorithmic choices affect convergence and robustness.

3. Algorithms

We solve the nonlinear least-squares problem [Hea18]

$$\min_{\beta \in \mathbb{R}^n} S(\beta), \quad S(\beta) = \frac{1}{2} \|r(\beta)\|_2^2 = \frac{1}{2} \sum_{i=1}^m r_i(\beta)^2,$$

where $r(\beta) \in \mathbb{R}^m$ is the residual vector and $J(\beta) \in \mathbb{R}^{m \times n}$ is its Jacobian with entries

$$J_{ij}(\beta) = \frac{\partial r_i(\beta)}{\partial \beta_j}.$$

Gauss-Newton (GN)

A useful identity for $S(\beta) = \frac{1}{2} \|r(\beta)\|_2^2 = \frac{1}{2} r(\beta)^T r(\beta)$ is that its gradient is

$$\nabla S(\beta) = J(\beta)^T r(\beta).$$

Differentiating once more yields a Hessian of the form

$$\nabla^2 S(\beta) = J(\beta)^T J(\beta) + \sum_{i=1}^m r_i(\beta) \nabla^2 r_i(\beta).$$

Here, the second term is multiplied by the residual components $r_i(\beta)$, which are expected to be small near a good fit. The Gauss-Newton method [Hea18] therefore drops this term and uses the approximation

$$\nabla^2 S(\beta) \approx J(\beta)^T J(\beta).$$

Applying a Newton-like step s_k at the current iterate β_k gives the normal equations

$$\left(J(\beta_k)^T J(\beta_k) \right) s_k = -J(\beta_k)^T r(\beta_k),$$

which can also be viewed as the $m \times n$ linear least-squares subproblem

$$J(\beta_k) s_k \cong -r(\beta_k).$$

In implementation, we solve this subproblem by an orthogonal factorization (e.g., QR) rather than forming $(J^T J)^{-1}$ explicitly for the numerical stability. We will discuss this more in the experiment section.

Pseudo-code (Gauss-Newton)

Input: initial guess β_0 , tolerances $(\varepsilon_{\text{step}}, \varepsilon_{\text{grad}})$, max iterations K .

For $k = 0, 1, \dots, K - 1$:

1. Compute $r_k = r(\beta_k)$ and $J_k = J(\beta_k)$.

2. Solve $J_k s_k \approx -r_k$ in the least-squares sense.
3. Update $\beta_{k+1} = \beta_k + s_k$.
4. Stop if $\|s_k\|_2 \leq \varepsilon_{\text{step}}(1 + \|\beta_k\|_2)$ or $\|J_k^T r_k\|_2 \leq \varepsilon_{\text{grad}}$.

Output: β_k .

$\varepsilon_{\text{step}}$ stops when the update s_k becomes very small relative to the current parameter scale, meaning the iterates aren't moving anymore. $\varepsilon_{\text{grad}}$ stops when $|J(\beta_k)^T r(\beta_k)|$ is small, meaning the objective's gradient is near zero and close to a stationary point.

Levenberg-Marquardt (LM)

Gauss-Newton can be unreliable when the linearized subproblem is ill-conditioned or when the initial guess is poor. The Levenberg-Marquardt modification [Hea18, Lev44, Mar63, Mor78], which introduces a damping parameter $\mu_k \geq 0$ and computes s_k from

$$\left(J(\beta_k)^T J(\beta_k) + \mu_k I \right) s_k = -J(\beta_k)^T r(\beta_k).$$

This can also be written as an augmented least-squares problem,

$$\begin{bmatrix} J(\beta_k) \\ \sqrt{\mu_k} I \end{bmatrix} s_k \approx \begin{bmatrix} -r(\beta_k) \\ 0 \end{bmatrix},$$

which highlights the regularization effect. When $\mu_k \rightarrow 0$, the method approaches Gauss-Newton; when μ_k is large, the step is closer to a scaled steepest-descent direction. μ_k dominates $J(\beta_k)^T J(\beta_k)$, so we approximate $J(\beta_k)^T J(\beta_k) + \mu_k I \approx \mu_k I$. Hence

$$\mu_k s_k = -J(\beta_k)^T r(\beta_k) \implies s_k \approx -\frac{1}{\mu_k} J(\beta_k)^T r(\beta_k).$$

Practically, μ_k is chosen by some strategy (e.g., a trust-region approach). Here, we use a simpler monotone decrease rule for clarity, which worked robustly in our regimes.

Pseudo-code (Levenberg-Marquardt).

Input: β_0 , initial damping $\mu_0 > 0$, factors $0 < \eta_{\downarrow} < 1 < \eta_{\uparrow}$, max iterations K .

For $k = 0, 1, \dots, K - 1$:

1. Compute $r_k = r(\beta_k)$ and $J_k = J(\beta_k)$.
2. Solve $(J_k^T J_k + \mu_k I) s_k = -J_k^T r_k$ (e.g., via QR on the augmented system).
3. Form the trial point $\beta_k^{\text{trial}} = \beta_k + s_k$.
4. **If** $S(\beta_k^{\text{trial}}) < S(\beta_k)$: accept $\beta_{k+1} = \beta_k^{\text{trial}}$ and set $\mu_{k+1} = \eta_{\downarrow} \mu_k$.
5. **Else**: reject the step, keep $\beta_{k+1} = \beta_k$, and set $\mu_{k+1} = \eta_{\uparrow} \mu_k$.

Output: final accepted β_k .

4. Experiments

We present three numerical experiments that examine how initialization, conditioning, and measurement noise affect the behavior of Gauss-Newton (GN) and Levenberg-Marquardt (LM). All tests use the damped oscillation model of Section 2, and all solvers are implemented in Python and NumPy. Unless otherwise noted, all solvers were run with a maximum of 80 iterations, a step tolerance $\varepsilon_{\text{step}} = 10^{-8}$, and a gradient tolerance $\varepsilon_{\text{grad}} = 10^{-6}$ (stopping when either criterion is met). For reproducibility, we fixed the random seed separately for each experiment.

E1: Sensitivity to Initialization

We generate data from

$$f(t; \beta) = Ae^{-\lambda t} \cos(\omega t + \phi) + c, \quad \beta_{\text{true}} = (1.3, 0.15, 2.0, 0.5, 0.1),$$

sampled at 200 uniformly spaced points in $[0, 10]$ with noise level $\sigma = 0.05$. To assess robustness, we generate 60 random initializations by applying independent Gaussian relative perturbations to each component of β_{true} , with clipping or wrapping to enforce physically meaningful ranges (e.g., $A > 0$, $\lambda > 0$, $\omega > 0$, $\phi \in [0, 2\pi)$). We also include 5 deliberately poor initial guesses (negative damping, incorrect amplitude sign, doubled frequency, and a point far from the truth). Each initialization is then solved with (i) undamped GN using normal equations and (ii) LM with $\mu_{\text{init}} = 10^{-3}$.

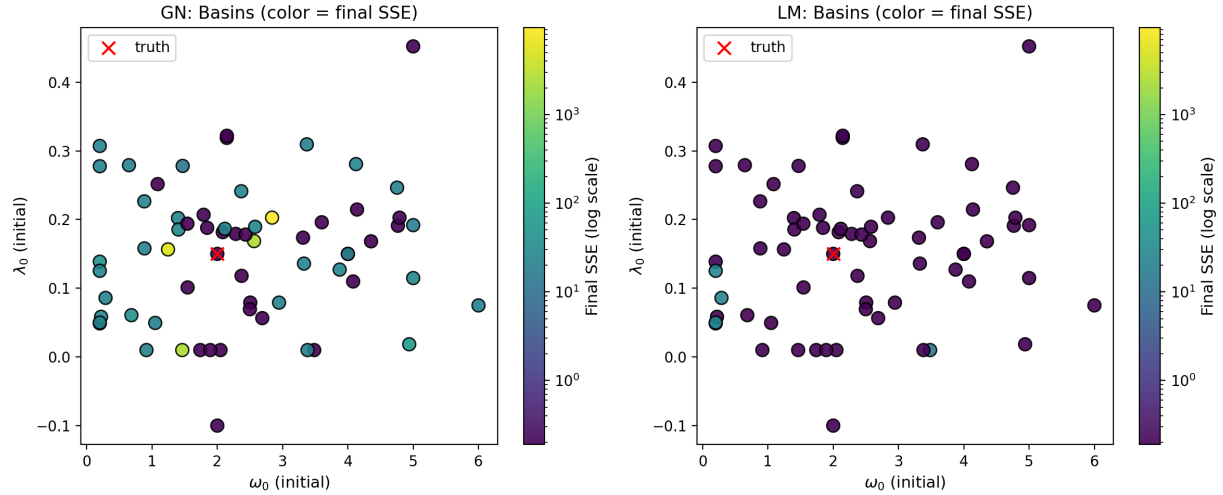


Figure 4.1: Sampled convergence outcomes for GN (left) and LM (right), colored by final SSE ($S(\beta) = \frac{1}{2}\|r(\beta)\|_2^2$) in log scale. LM maintains a substantially larger region of reliable convergence.

Figure 4.1 visualizes convergence outcomes over the (ω_0, λ_0) plane of initial guesses. Each point is colored by the final SSE (darker indicates smaller SSE), allowing a direct comparison

of GN and LM. GN exhibits widespread failures and relatively high-SSE outcomes. When residuals are not small, GN’s Hessian approximation can be inaccurate, which may cause stagnation or divergence (see Section 5). In contrast, LM converges successfully across a much broader range of initializations and achieves consistently low SSE.

Figure 4.2 summarizes the convergence outcomes: out of 65 initializations, GN converges in 22 cases, while LM converges in 60 cases. It also includes a representative run in which GN stagnates, whereas LM decreases the objective smoothly. Overall, these results show that GN converges reliably only from a limited range of initial guesses, while LM remains robust across a much broader set of starting points.

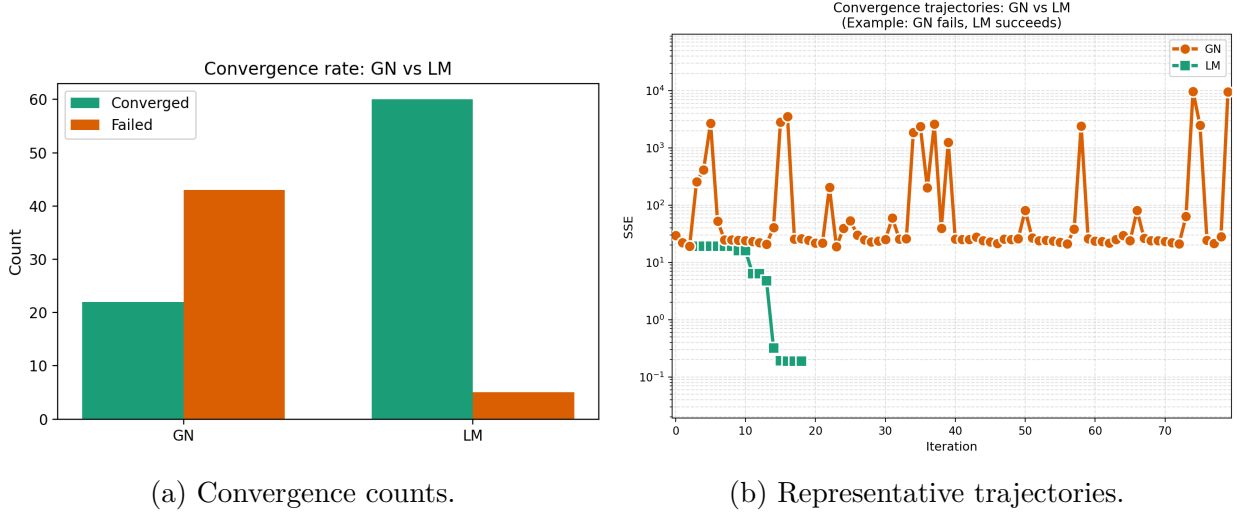


Figure 4.2: (a) GN succeeds on fewer initializations, whereas LM converges reliably. (b) An example that GN diverges by the 80-iteration limit, whereas LM converges in 18 iterations.

E2: Ill-Conditioning of the Subproblem

To examine conditioning effects, we extend the time horizon to $[0, 25]$ with 320 samples using

$$\beta_{\text{true}} = (0.8, 0.08, 1.6, 0.9, 0.05), \quad \beta_0 = (1.2, 0.02, 1.0, 0.1, -0.2).$$

At large t , the factor $e^{-\lambda t}$ damps the signal toward a near-constant baseline, so the sensitivities with respect to several parameters become similar and the columns of J approach linear dependence.

We compare three solvers (algorithms discussed in Section 3): (i) GN with the update from the normal equations $J^\top J s_k = -J^\top r$. (ii) GN with a QR factorization of J . (iii) LM with adaptive damping (dynamically adjusted μ). Figure 4.3 reports condition numbers over the iterations. Even though $\text{cond}(J)$ stays moderate, both GN variants show a sharp increase in $\text{cond}(J^\top J)$, reaching up to 10^{12} . This aligns with the theoretical relation $\text{cond}(J^\top J) = \text{cond}(J)^2$. LM maintains a well-conditioned system by lifting the smallest singular values of $J^\top J$.

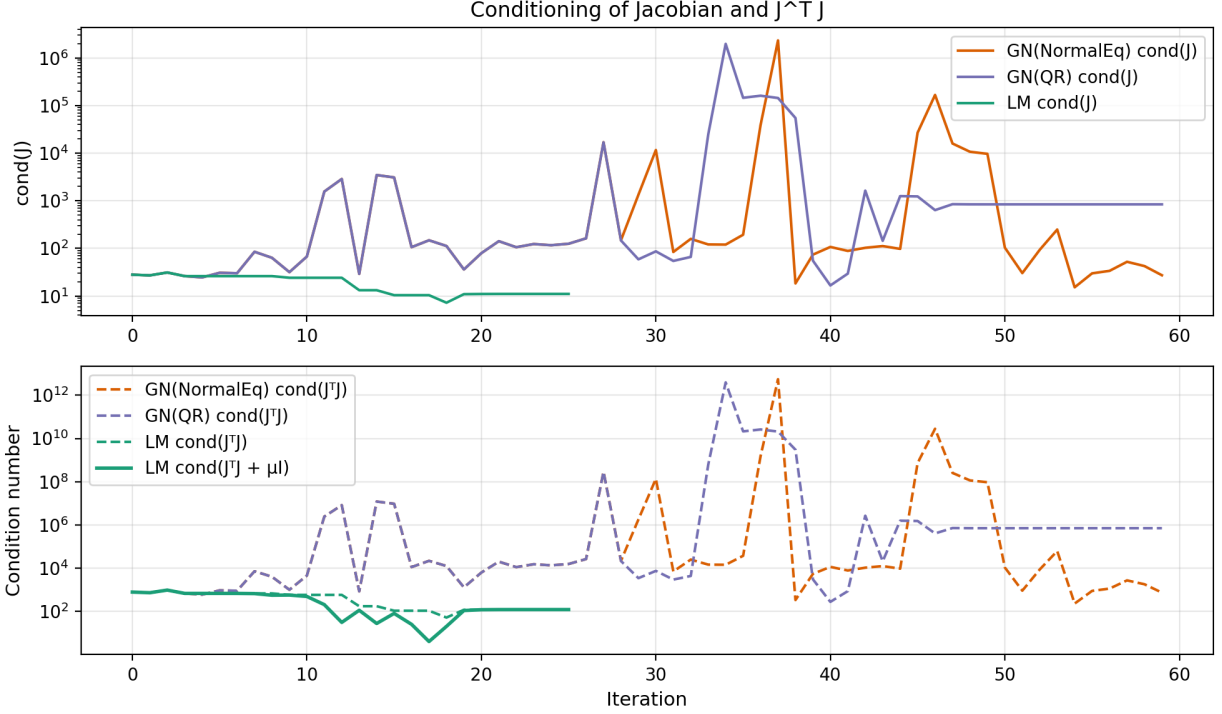


Figure 4.3: Conditioning of J and $J^T J$. GN methods suffer from severe ill-conditioning, while LM keeps the regularized matrix $J^T J + \mu I$ well conditioned.

The SSE trajectories in Figure 4.4 highlight the practical consequences of ill-conditioning. GN (normal eq) displays instability and objective spikes. Solving the subproblem by QR mitigates the linear-solve instability from forming $J^T J$, but GN remains unreliable here. In contrast, LM yields a smooth, steady decrease and reaches the lowest final SSE by regularizing the system throughout the iterations.

E3: Sensitivity to Measurement Noise

At last, we analyze how measurement noise affects parameter recovery. Using the parameters of E1, we fix a mildly biased initialization

$$\beta_0 = (0.9A, 1.1\lambda, 0.8\omega, \phi + 0.2, c)$$

and consider noise levels $\sigma \in \{0.01, 0.05, 0.10, 0.20\}$. For each σ , twenty noisy datasets are generated, GN is run to convergence, and we record the relative parameter error and final SSE (medians and interquartile ranges). We focus on GN because this experiment isolates the noise-to-parameter-error relationship under favorable conditioning; LM behaves similarly here.

Figure 4.5 shows that the parameter error, measured by the relative ℓ_2 error $\frac{\|\hat{\beta} - \beta_{true}\|_2}{\|\beta_{true}\|_2}$, increases approximately linearly with σ , consistent with first-order perturbation theory when

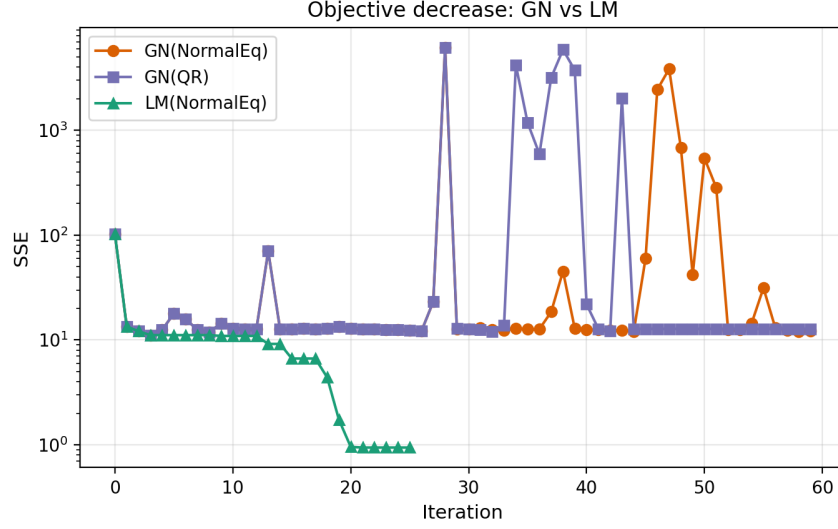


Figure 4.4: SSE versus iteration. Ill-conditioning destabilizes GN, whereas LM converges smoothly.

the Jacobian is well conditioned. Final SSE grows more steeply, approximately like $O(\sigma^2)$. It reflects both irreducible noise energy, which scales with the noise variance, and residual model mismatch.

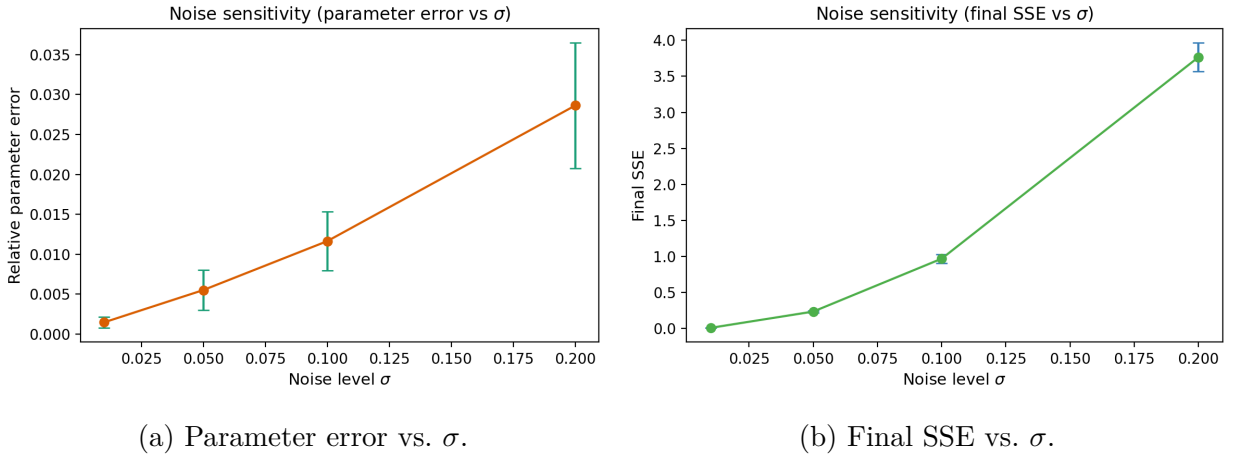


Figure 4.5: Sensitivity to measurement noise.

Across all noise levels considered, GN remains stable so long as the underlying problem is well conditioned and initialized near the true solution.

5. Discussion

The experiments highlight complementary aspects of the numerical behavior of Gauss-Newton (GN) and Levenberg-Marquardt (LM). GN is effective only in favorable regimes, while LM remains reliable when those assumptions are weakened. Quantitatively, in E1 GN converges on $\frac{22}{65}$ initializations versus $\frac{60}{65}$ for LM. In E2 $\text{cond}(J^\top J)$ for GN reaches 10^{12} . In E3 the relative parameter error grows roughly linearly with σ while the final SSE scales approximately like $O(\sigma^2)$.

Initialization (E1) GN’s narrow basin of attraction is consistent with its Hessian approximation $\nabla^2 S(\beta) = J^\top J + \sum_i r_i \nabla^2 r_i \approx J^\top J$: for poor starting points the residuals are not small, the omitted term matters, and the resulting steps can be inaccurate (often non-descent). LM’s damping stabilizes early iterations, leading to far fewer failures in the same initialization set.

Conditioning (E2) When the columns of J become nearly dependent, forming the normal equations squares the conditioning and makes the GN subproblem numerically fragile, consistent with the observed growth of $\text{cond}(J^\top J)$ up to 10^{12} . LM counters this by solving a regularized system $J^\top J + \mu I$, which keeps the linear algebra well behaved and produces stable decreases in the objective.

Noise (E3) Under good conditioning and a mildly biased start, noise mainly affects estimator variance rather than convergence: the relative error $\frac{\|\hat{\beta} - \beta_{\text{true}}\|_2}{\|\beta_{\text{true}}\|_2}$ increases approximately linearly with σ , while the final SSE grows more steeply (about $O(\sigma^2)$) because it reflects irreducible noise energy in addition to any model mismatch.

Cost LM’s added overhead is modest: each iteration solves a linear system of the same dimension as GN (often via the same QR-based machinery, e.g., through an augmented system) and performs a simple acceptance test to update μ_k , in exchange for substantially improved robustness.

6. Conclusion

We investigated three factors that strongly influence the performance of nonlinear least-squares solvers: initialization, conditioning, and measurement noise. Using a representative damped oscillation model, we compared the behavior of Gauss-Newton (GN) and Levenberg-Marquardt (LM) across a range of challenging scenarios.

Across all experiments, LM demonstrated markedly greater robustness: it converged from a broad set of initial guesses, maintained stable progress when the Jacobian became ill-conditioned, and behaved predictably under increasing noise. GN, while efficient under ideal conditions, remained highly sensitive to initialization and significantly degraded when

the normal equations became ill-conditioned. Under favorable conditioning, convergence behavior stayed predictable, but estimation accuracy degraded: relative parameter error increased roughly linearly with σ , while the final SSE grew more steeply, reflecting the contribution of noise variance.

Overall, GN is attractive when a good initialization and benign conditioning can be assumed, but LM is the safer default when either is uncertain, offering substantially improved reliability at modest extra per-iteration cost.

Our analysis is based on a single model and a squared-loss objective applied to synthetic data, and therefore does not fully account for model mismatch or real-world measurement artifacts. These limitations point to natural extensions of this work, such as incorporating globalization strategies for GN (e.g., line search or trust-region methods), exploring robust loss functions such as the Huber penalty, and validating the solvers on a small real dataset.

References

- [Hea18] M. T. Heath. *Scientific Computing: An Introductory Survey*. 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2018.
- [Lev44] K. Levenberg. A method for the solution of certain non-linear problems in least squares. *Quarterly of Applied Mathematics*, 2(2): 164–168, 1944.
- [Mar63] D. W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2): 431–441, 1963.
- [Mor78] J. J. Moré. The levenberg-marquardt algorithm: implementation and theory. *Numerical Analysis*. Ed. by G. A. Watson. Berlin, Heidelberg: Springer, 1978, pp. 105–116.
- [Wik25a] Wikipedia contributors. *Gauss–Newton algorithm* — *Wikipedia, The Free Encyclopedia*. 2025.
- [Wik25b] Wikipedia contributors. *Harmonic oscillator* — *Wikipedia, The Free Encyclopedia*. 2025.
- [Wik25c] Wikipedia contributors. *Levenberg–Marquardt algorithm* — *Wikipedia, The Free Encyclopedia*. 2025.

A. Code Availability

All code used to generate the experiments and figures in this report is publicly available at: <https://github.com/TianX-IA/AM205Project.git>